pycon.fr
BIARRITZ 2026
29 octobre - 1er novembre

NAPTA

Migrer les données d'une application Web… sans que tout explose

Loïc Simon

Qui suis-je

/in/loic-simon

ESPCI PARIS | PSL

MINES PARIS | PSL

NAPTA

Senior Staff Engineer
Mars 2022 —

/loic-simon

4 packages sur PyPI
Contributions à CPython (~15 PR),
mypy, SQLAlchemy...

À propos de Napta

Le système de resource management leader en Europe, **plébiscité par les plus grandes sociétés de services professionnels.**

Fondée en 2017

Rachetée par

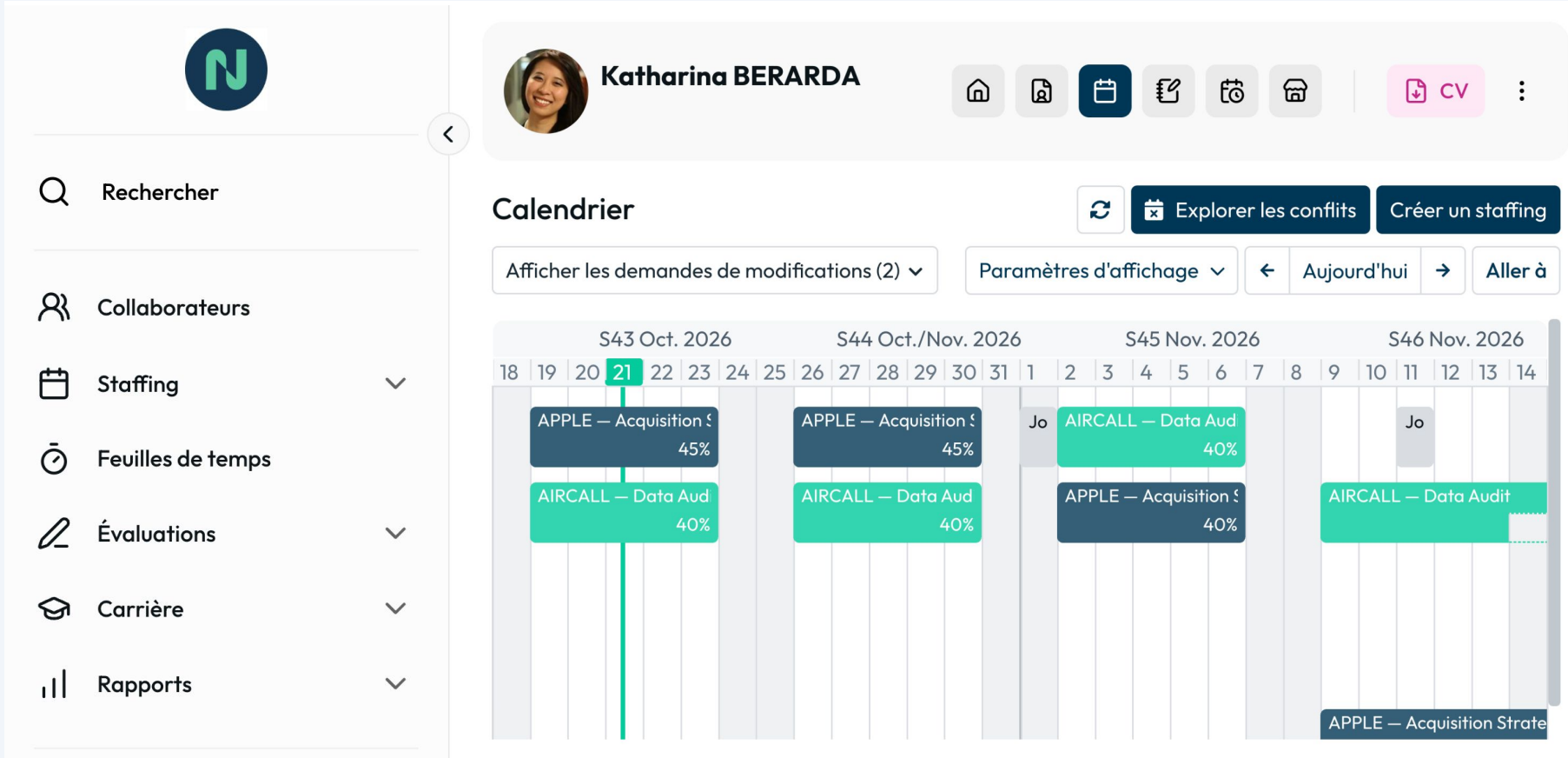
UNIT4

en octobre 2026

sopra steria · interpath · Roland Berger · KPMG · Grant Thornton · pwc

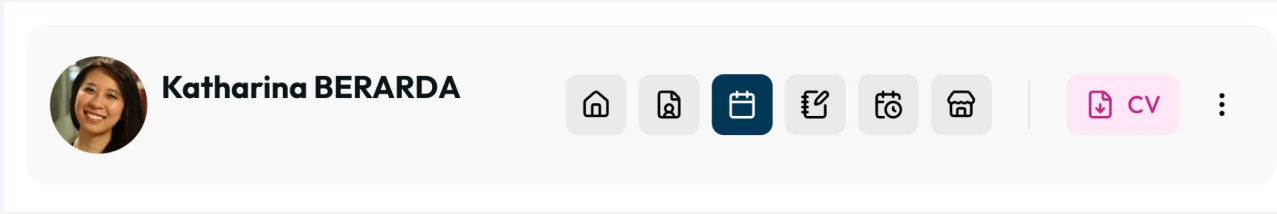
- Solution utilisée quotidiennement par plus de 150 entreprises dans 70 pays
- Hébergé en Europe, certifié ISO 27001 et appuyé sur des partenaires technologiques solides (AWS, Okta et OpenAI)
- Solution augmentée par l'IA, 100% sécurisée
- +10 ans d'expérience dans l'industrie des sociétés de services (et ça continue)
- Totalement intégré à votre système d'information
- Partenariats solides avec des outils de référence internationaux (Workday & SAP Partner)

NAPTA en un schéma



NAPTA en un schéma





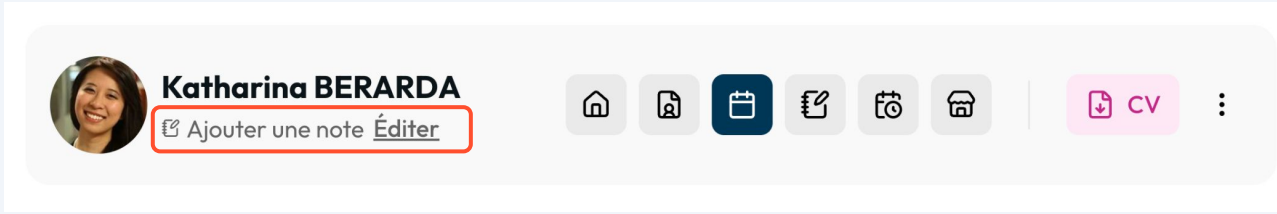

```python
class User(BaseModel):
    __tablename__ = "user"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str]
```

```sql
CREATE TABLE public.user (
    id SERIAL,
    name VARCHAR NOT NULL,

    CONSTRAINT user_pkey PRIMARY KEY (id)
);
```

NAPTA en un schéma






```python
class User(BaseModel):
    __tablename__ = "user"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str]
    notes: Mapped[str | None]
```

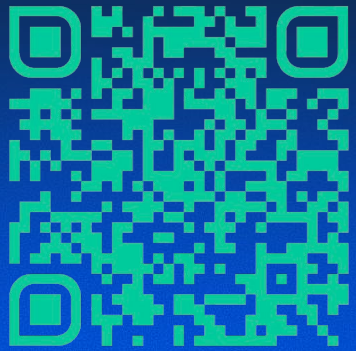
```sql
CREATE TABLE public.user (
    id SERIAL,
    name VARCHAR NOT NULL,
    notes VARCHAR NULL,
    CONSTRAINT user_pkey PRIMARY KEY (id)
);
```

Au programme

- **Migrer le schéma** sans downtime ?
- **Migrer la donnée** sans pertes ?
- **Assurer l'intégrité** des migrations ?

Retrouvez les diapos et le code :

loic-simon.github.io



Migrer le schéma avec Alembic

```
$ git diff
--- a/app/models/user.py
+++ b/app/models/user.py
@@ -164,6 +164,7 @@ class User(BaseModel):
     name: Mapped[str]
+    notes: Mapped[str | None]
```

```
$ alembic revision -m "Add notes" --autogenerate
INFO  [alembic.autogenerate.compare.tables] Detected added
column 'user.notes'
Generating app/alembic/versions/1975ea83b712_add_notes.py
...done
```

Migrer le schéma avec Alembic

```
$ cat app/alembic/versions/1975ea83b712_add_notes.py
revision = '1975ea83b712'         # N+1
down_revision = '9dd8a0bcabe8'    # N

from alembic import op
import sqlalchemy as sa

def upgrade():
    op.add_column(
        'user',
        sa.Column('notes', sa.String())
    )

def downgrade():
    op.drop_column('user', 'notes')
```

Migrer le schéma avec Alembic

```
$ psql -c "SELECT * FROM public.user;"
 id |       name
----+------------------
  1 | Katharina BERARDA
(1 row)

$ psql -c "SELECT * FROM public.alembic_version;"
 version_num
--------------
 9dd8a0bcabe8
(1 row)

$ alembic upgrade head
INFO  [alembic] Running upgrade 9dd8a0bcabe8 -> 1975ea83b712
```

Migrer le schéma avec Alembic

```
$ psql -c "SELECT * FROM public.user;"
 id |       name         | notes
----+--------------------|-------
  1 | Katharina BERARDA |
(1 row)

$ psql -c "SELECT * FROM public.alembic_version;"
 version_num
--------------
 1975ea83b712    ⇒ N+1
(1 row)

$ alembic upgrade head
INFO  [alembic] Nothing to do
```

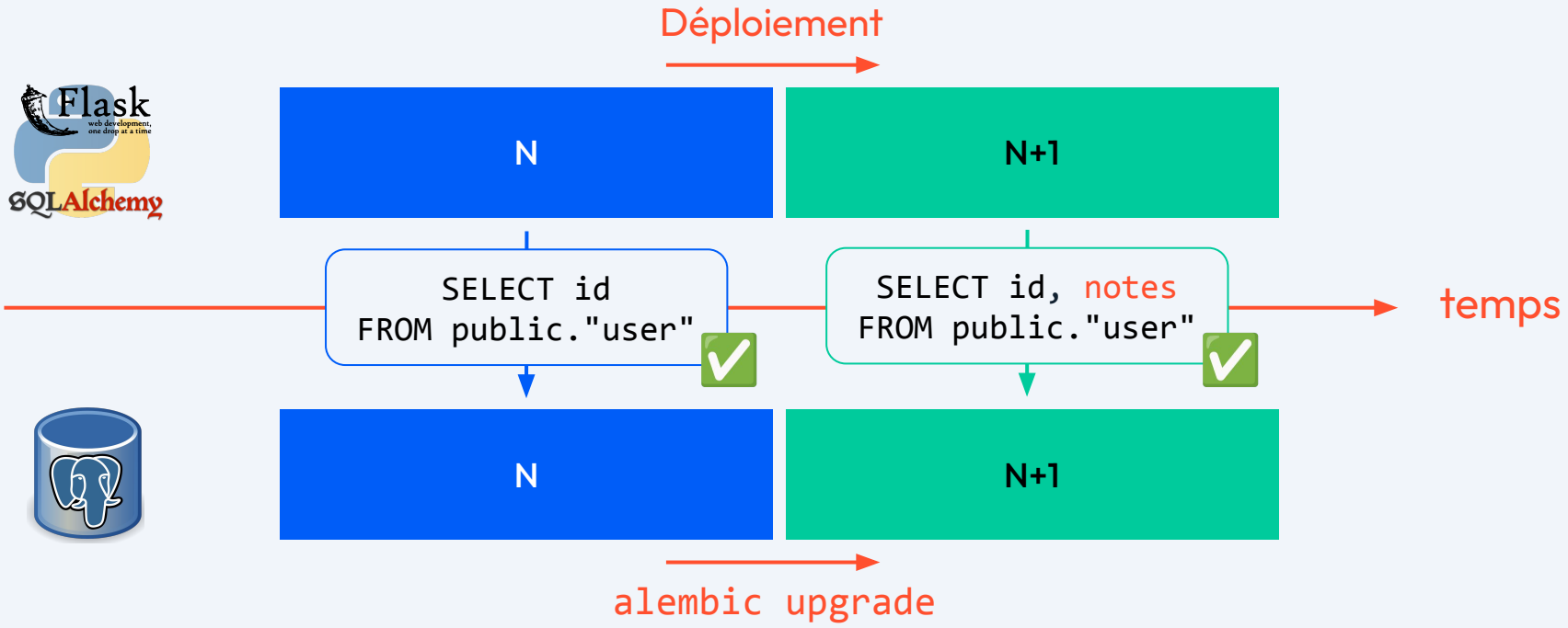
Migrer le schéma avec Alembic

```
$ cat alembic/env.py
def run_migrations_online() -> None:
    engine = engine_from_config(
        config.get_section(config.config_ini_section, {}),
        prefix="sqlalchemy.",
        poolclass=pool.NullPool,
    )

    with engine.connect() as connection:
        context.configure(
            connection=connection,
            target_metadata=BaseModel.metadata,
        )
        with context.begin_transaction():
            context.run_migrations()
```

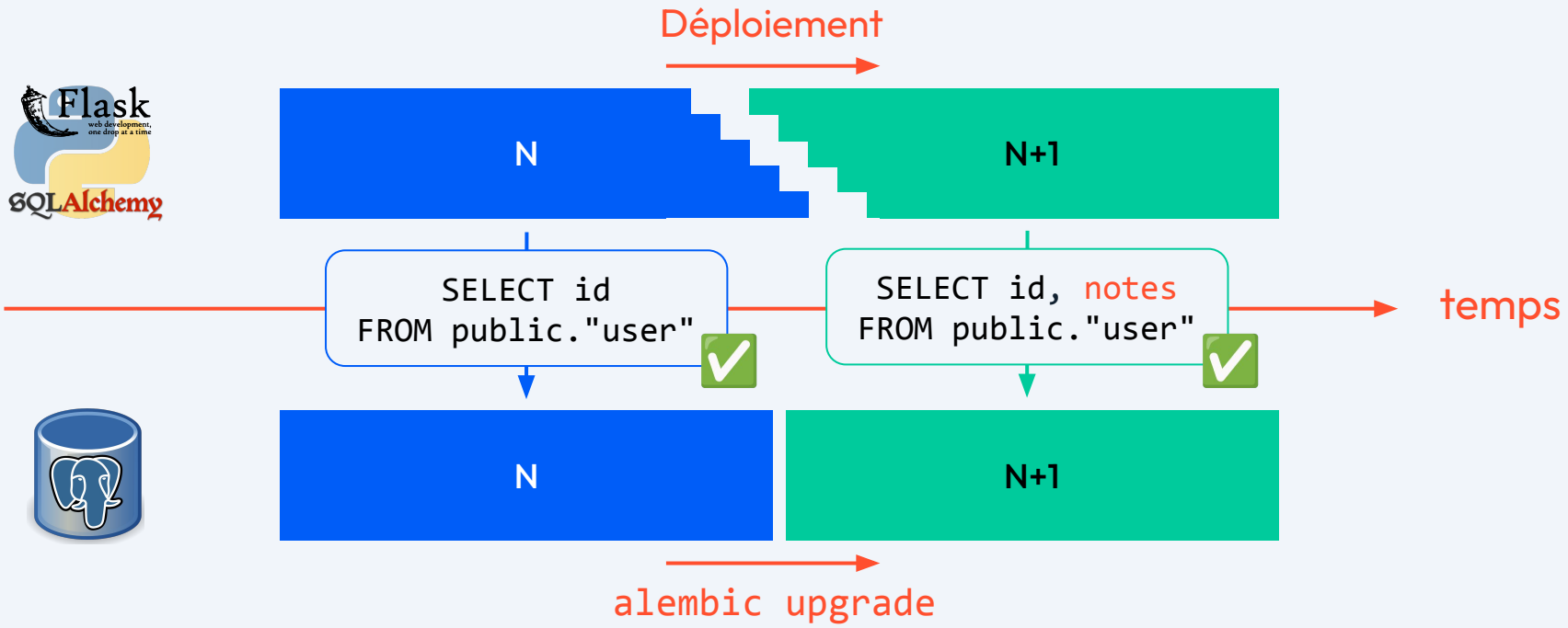
Quand appliquer la migration ?

Passage de la version **N** *(pas de notes)* à la version **N+1** *(notes)*



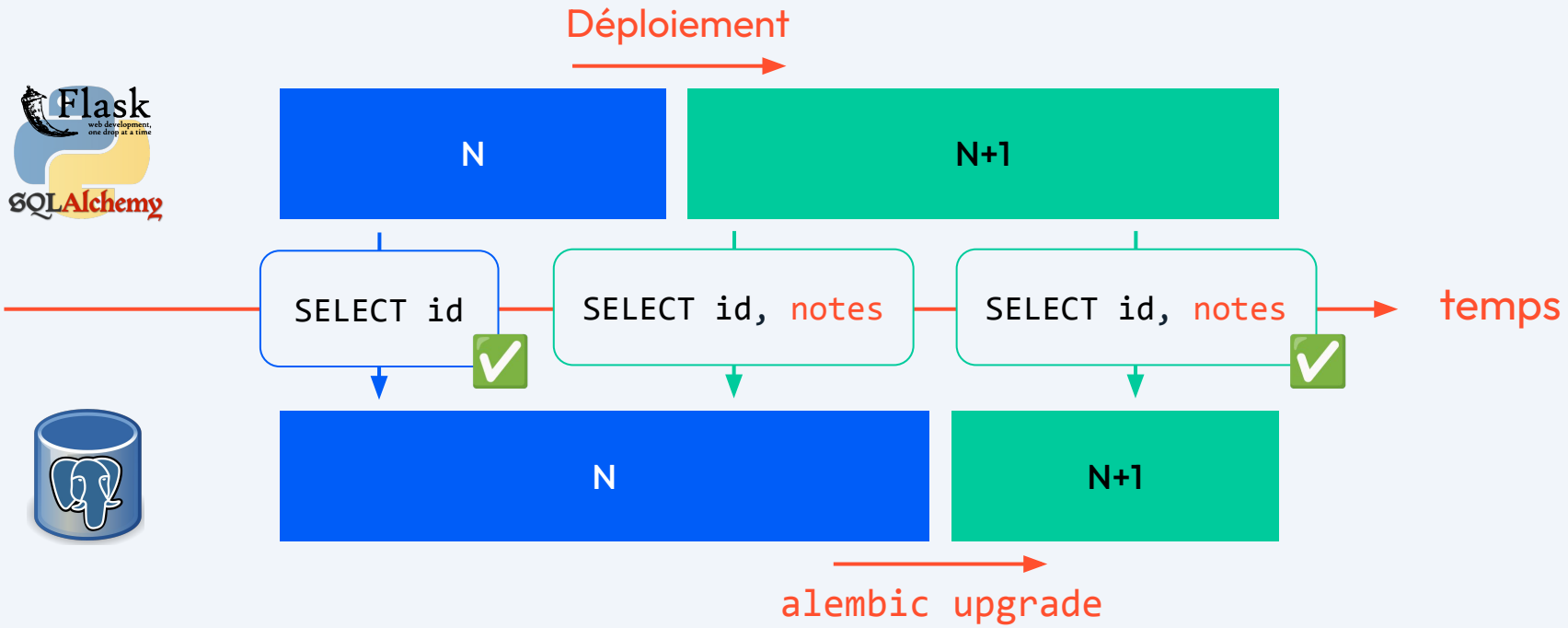
Quand appliquer la migration ?

Ça ne peut pas être exactement en même temps...



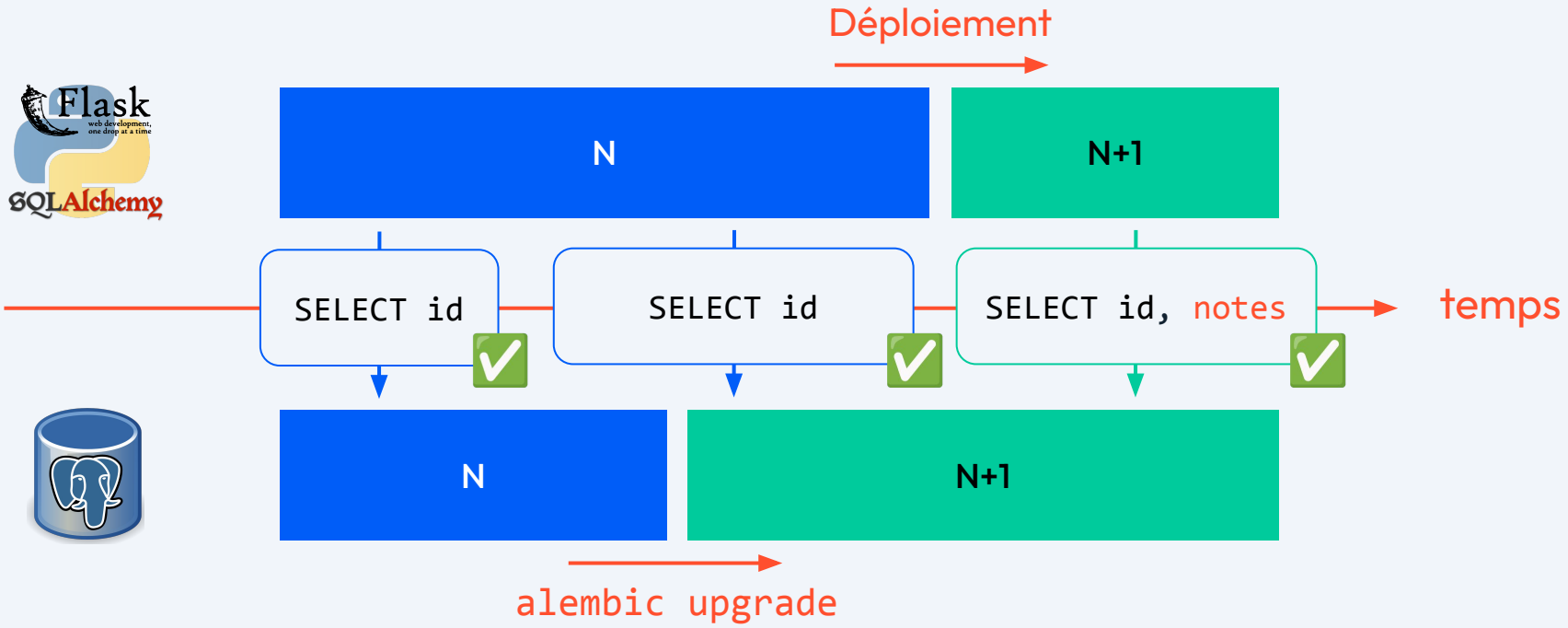
Quand appliquer la migration ?

Ça ne peut pas être exactement en même temps...



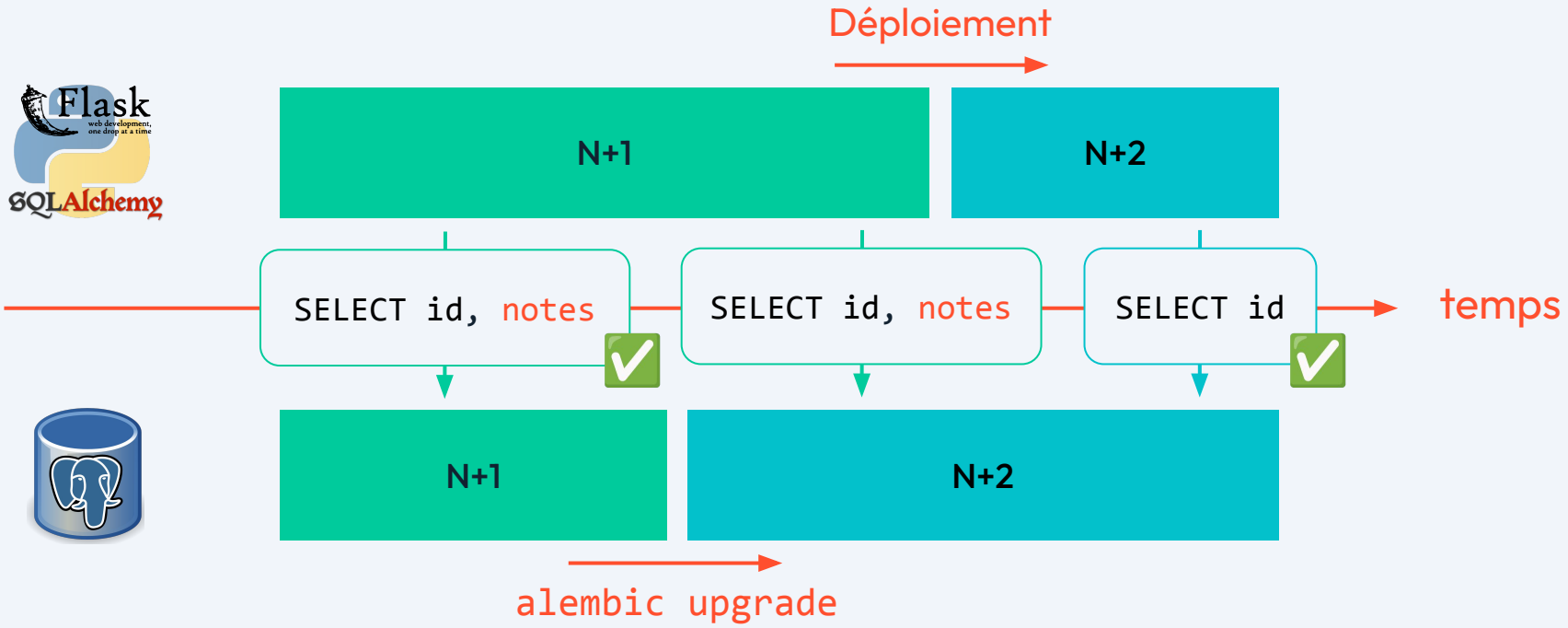
Quand appliquer la migration ?

Ça ne peut pas être exactement en même temps...



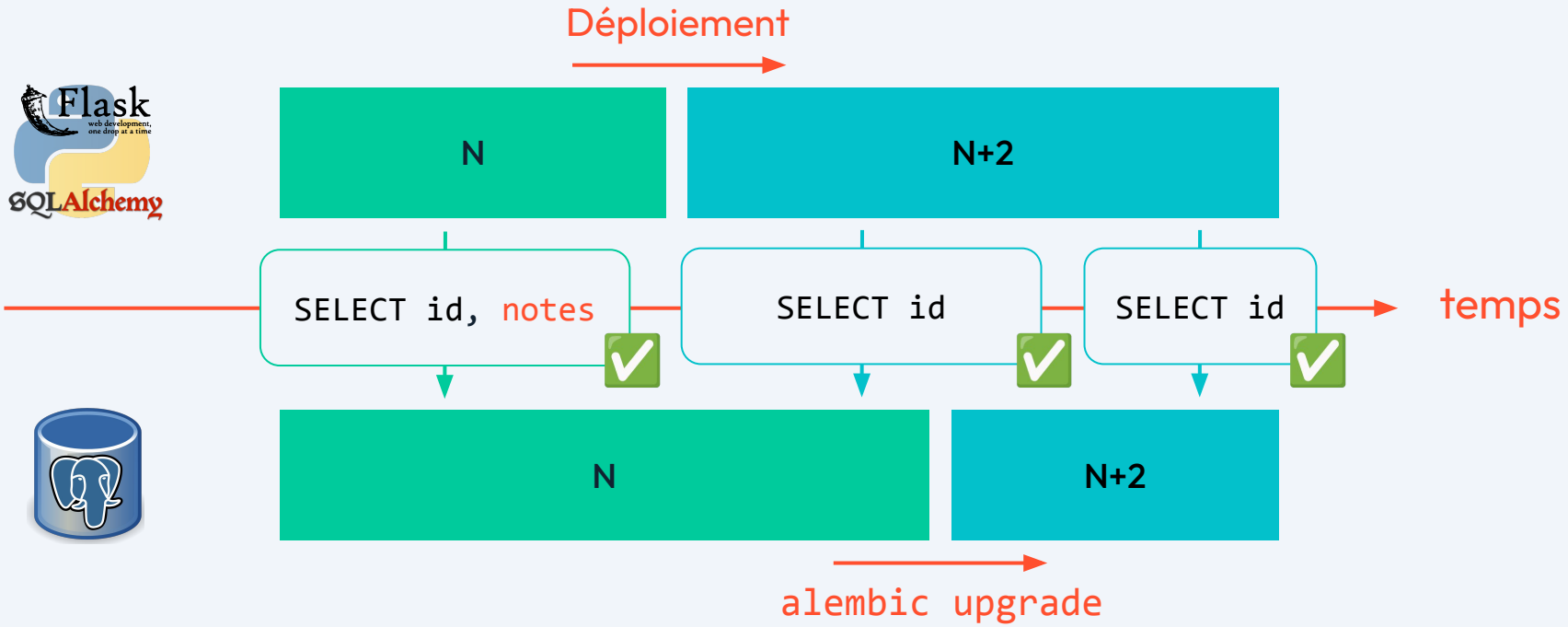
Quand appliquer la migration ?

... mais imaginons la migration inverse ! Version **N+2** *(suppression de user.notes)*



Quand appliquer la migration ?

… mais imaginons la migration inverse ! Version **N+2** *(suppression de user.notes)*



Quand appliquer la migration ?

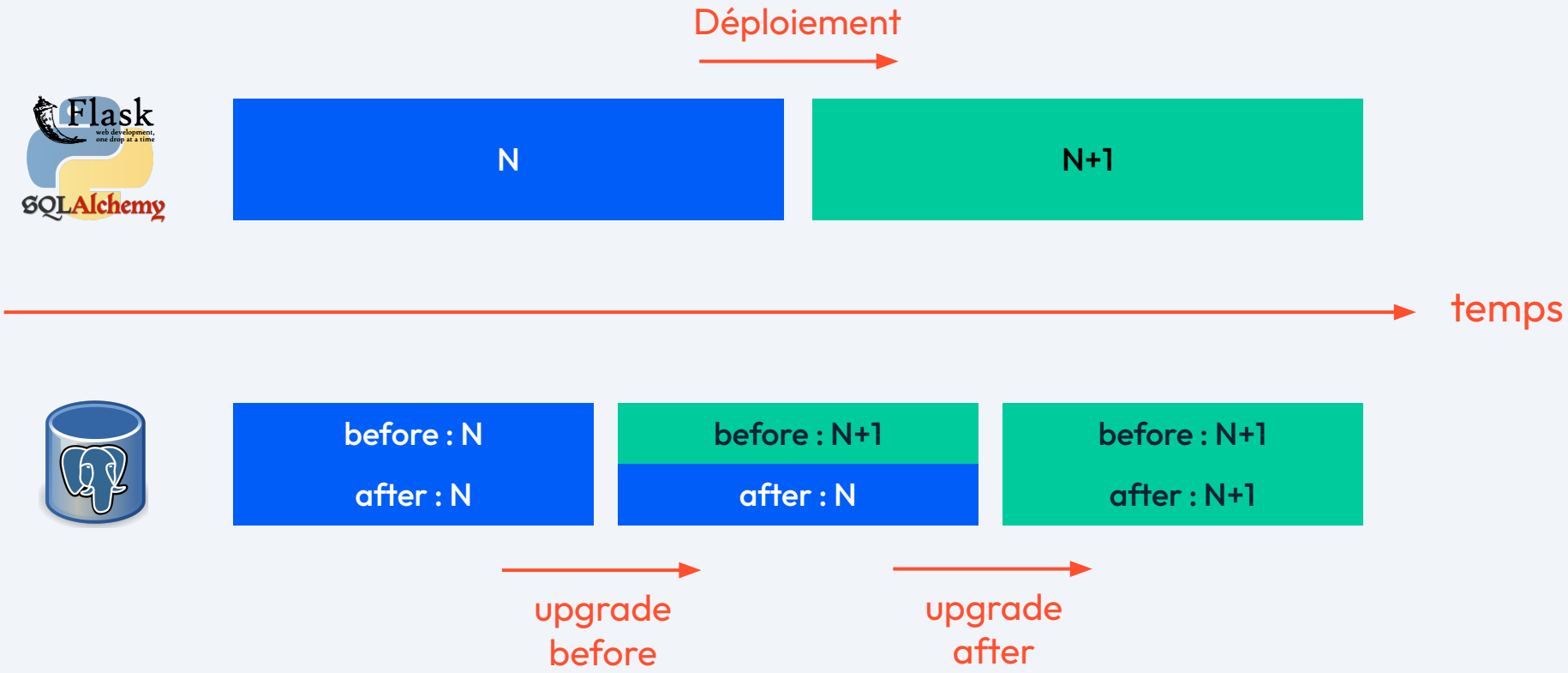
En résumé :

Ordre de migration	**Upgrade puis déploiement** (“upgrade-before”)	**Déploiement puis upgrade** (“upgrade-after”)
Nouvelle colonne	OK ✅	Downtime 💣
Suppression de colonne	Downtime 💣	OK ✅

Solution : **adapter l’ordre aux opérations** de la migration

⇒ Chaque migration est **décomposée en deux étapes**
⇒ Framework de **migration en deux temps**

Migration en deux temps



Migration en deux temps

```
$ psql -c "CREATE TABLE alembic_version_after (version_num TEXT);"

$ cat alembic/env.py
...
    step: Literal["before", "after"] = os.getenv("MIGRATION_STEP", "before")
    table = "alembic_version_after" if step == "after" else "alembic_version"

    with engine.connect() as connection:
        context.configure(
            connection=connection,
            target_metadata=BaseModel.metadata,
            version_table=table
        )
        with context.begin_transaction():
            context.run_migrations(step=step)
```

Migration en deux temps

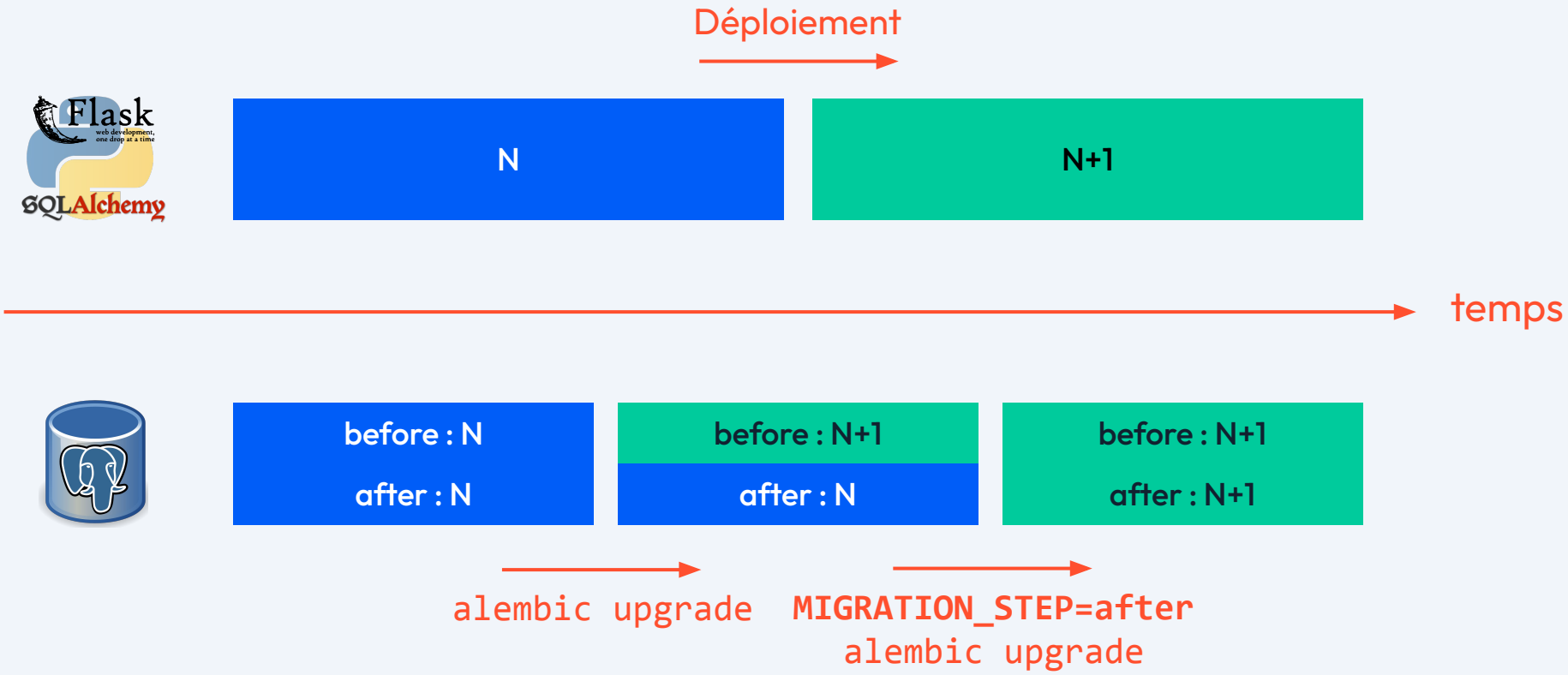
```
$ cat 1975ea83b712_add_notes.py
revision = '1975ea83b712'        # N+1
down_revision = '9dd8a0bcabe8'   # N

def upgrade(*, step: Literal["before", "after"]):
    if step == "before":
        op.add_column("user", sa.Column("notes", sa.String()))

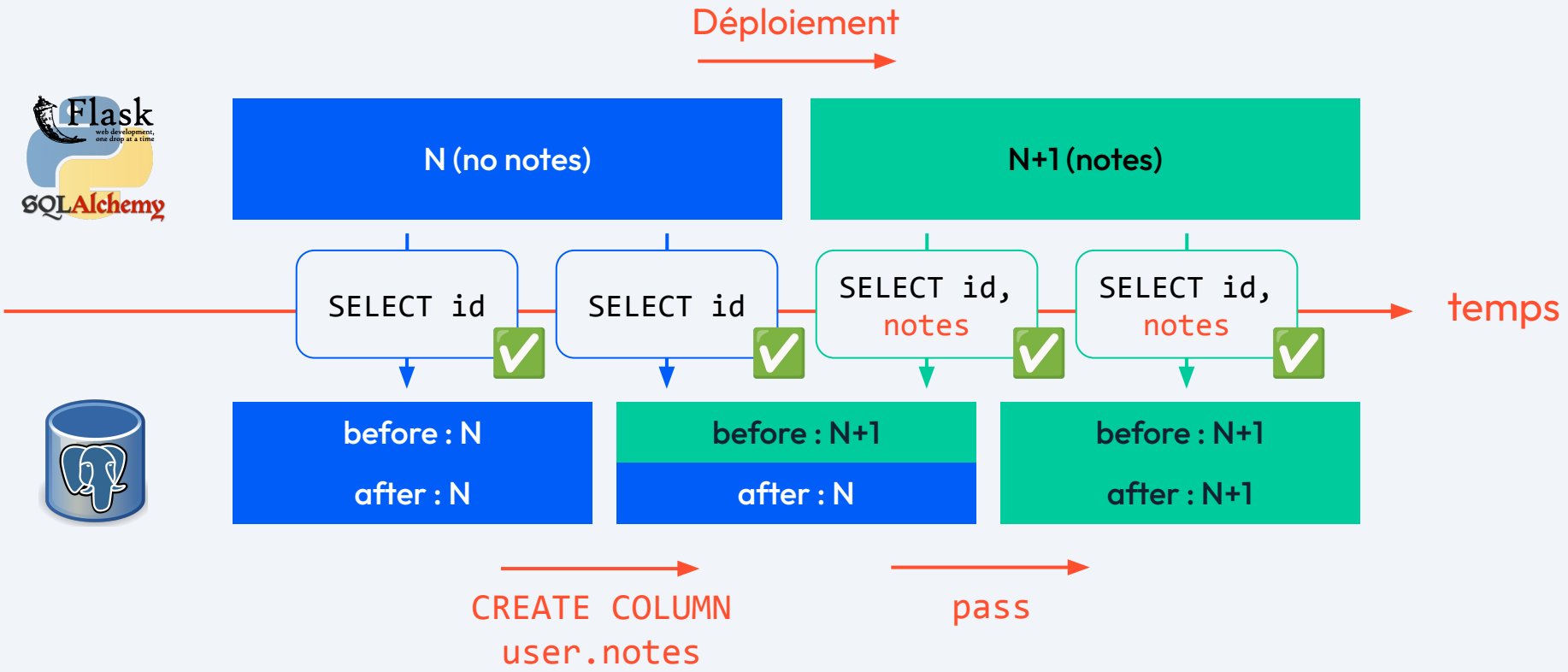

$ cat 9cb7b109ec9b_remove_notes.py
revision = '9cb7b109ec9b'        # N+2
down_revision = '1975ea83b712'   # N+1

def upgrade(*, step: Literal["before", "after"]):
    if step == "after":
        op.drop_column("user", "notes")
```

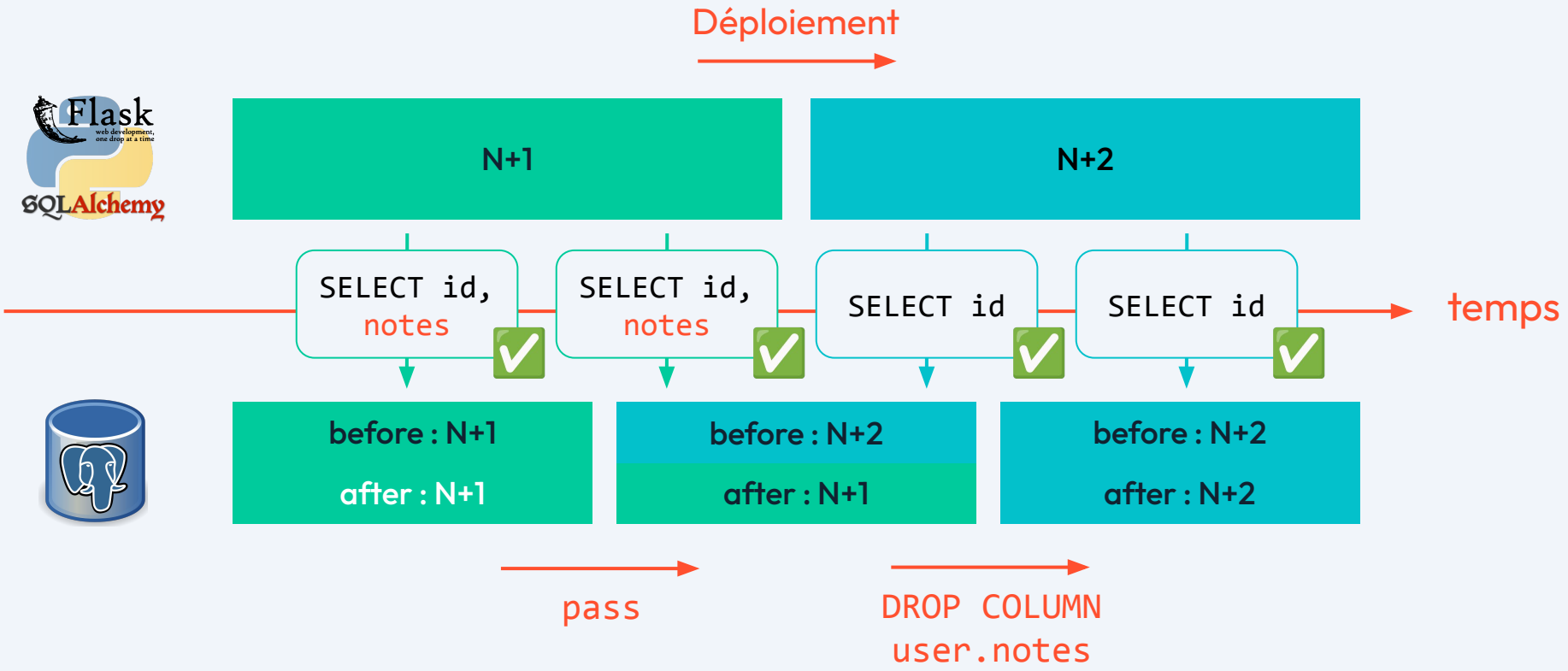
Migration en deux temps



Migration en deux temps



Migration en deux temps



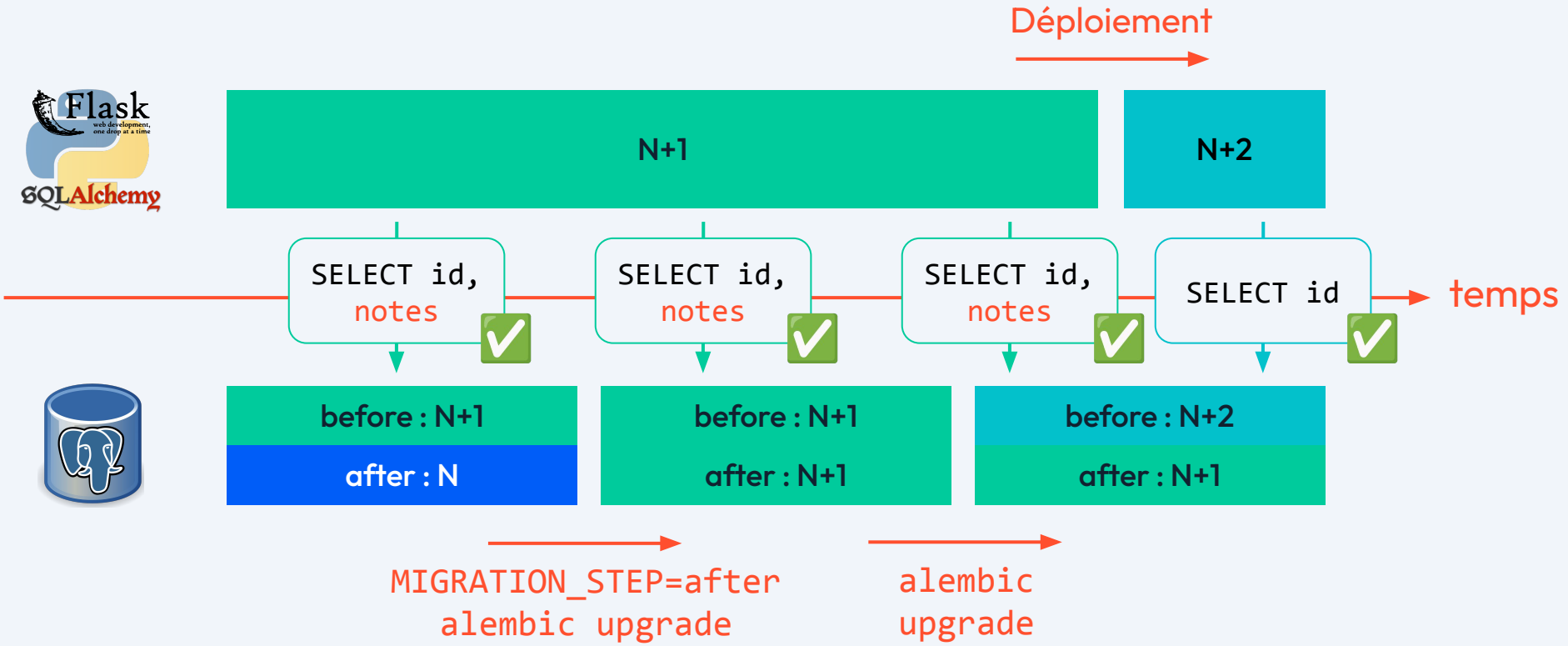
Points d’attention

Quelques complexités liées au workflow :

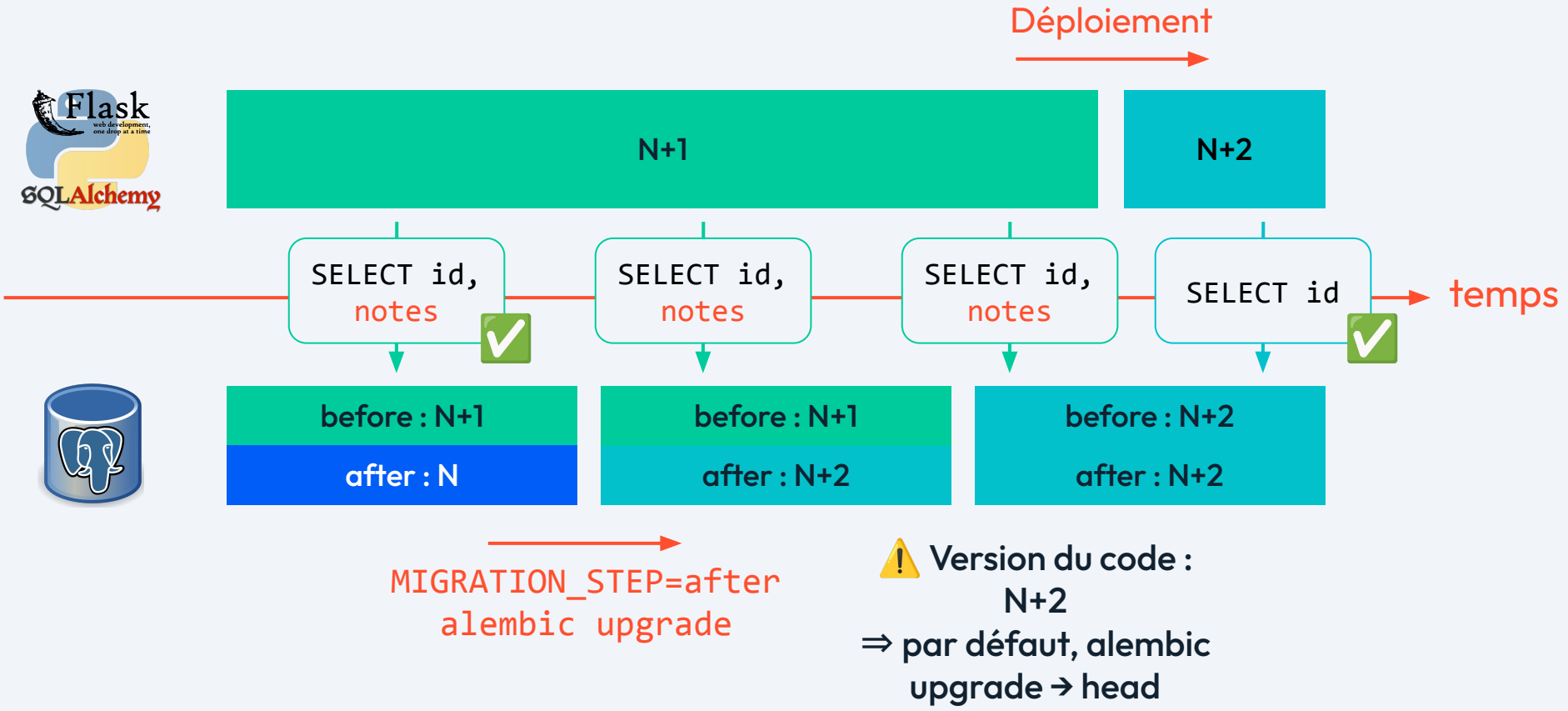
- **Downgrades** :
  - même problématique → pas envie de tout écrire en double
  - pas utilisés en production
  - une deuxième migration viendra annuler la première si nécessaire

- **Rollbacks de déploiement** :
  - possibles tant que l’after n’est pas passé
  - ne pas passer l’after avant **le prochain déploiement**

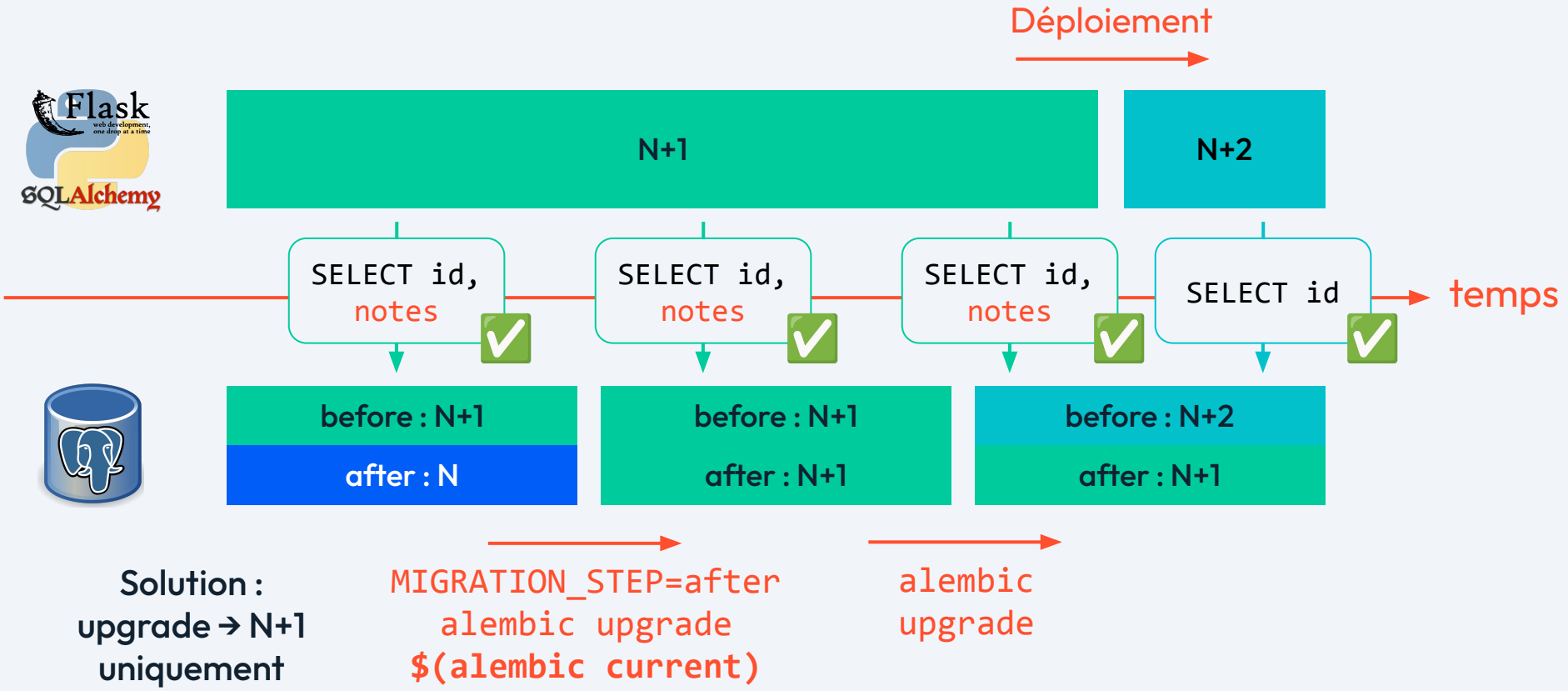
Points d'attention



Points d'attention



Points d'attention



Points d’attention

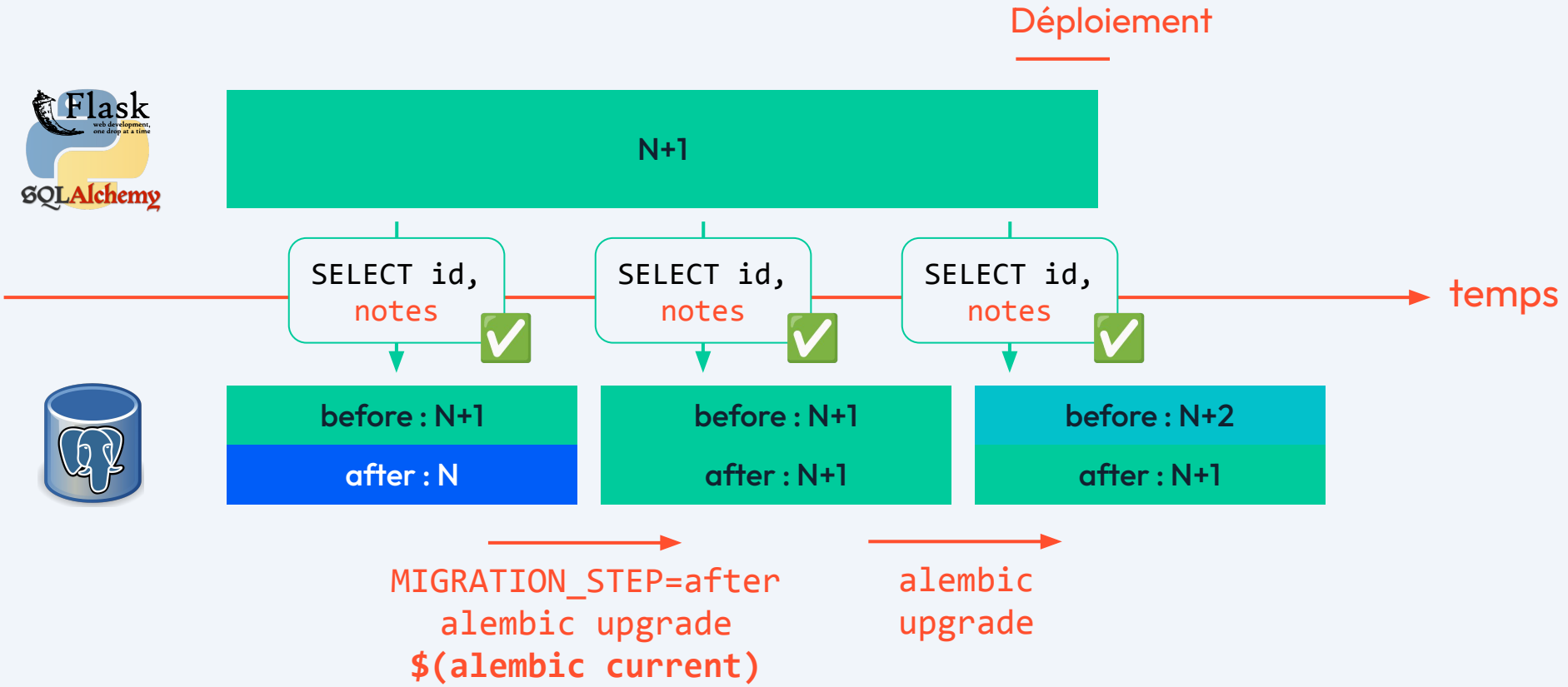
Quelques complexités liées au workflow :

- **Downgrades** :
  - même problématique → pas envie de tout écrire en double
  - pas utilisés en production *(forward-only)*
  - une deuxième migration viendra annuler la première si nécessaire

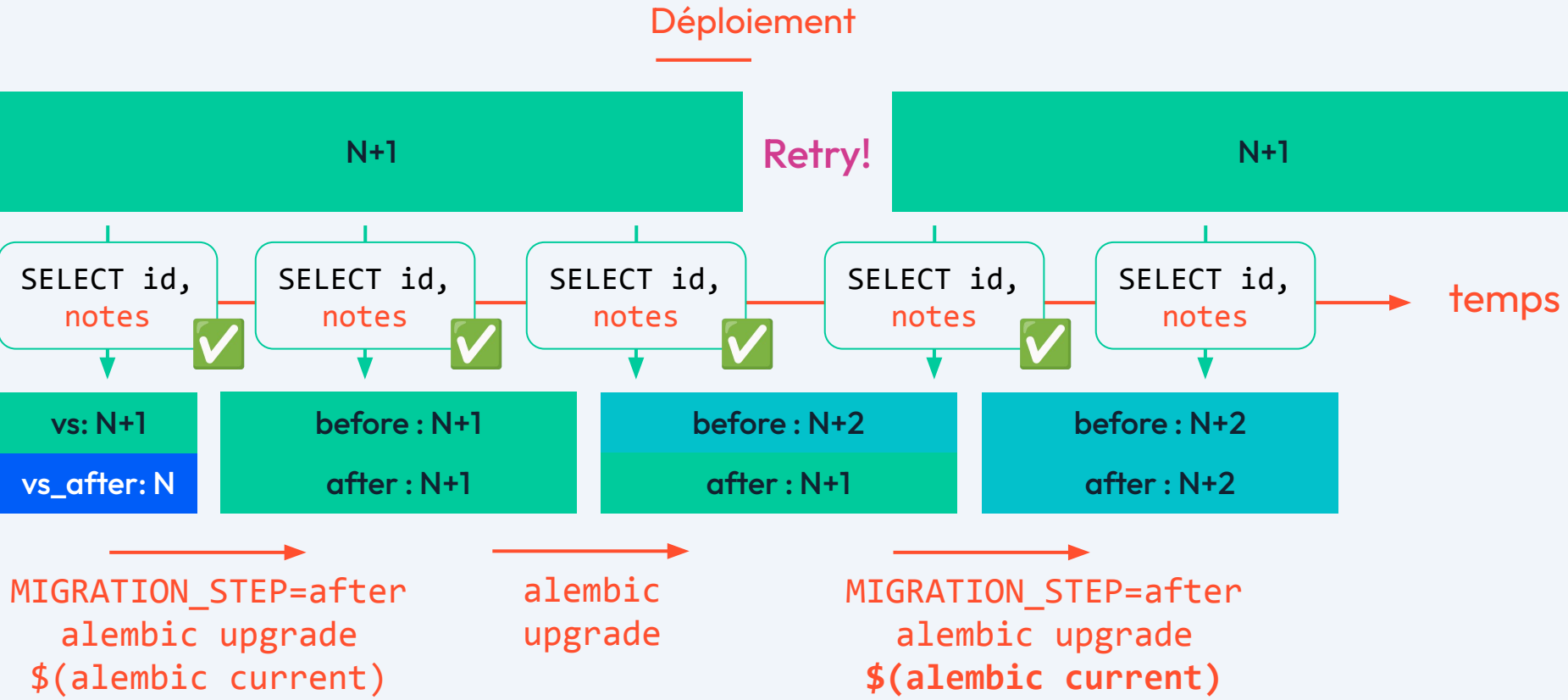
- **Rollbacks de déploiement** :
  - possibles tant que l’after n’est pas passé
  - ne pas passer l’after avant **le prochain déploiement**

- **Retries de déploiement** :
  - attention à ne pas mettre la charrue avant les boeufs

Points d'attention



Points d'attention



Points d’attention

Solution :

- Ajout d’une colonne **deployment_tag** (integer)
  aux tables **alembic_version** & **alembic_version_after**

- Chaque déploiement a un **tag dédié**, **supérieur** au précédent

- **Après chaque upgrade** (before et after), on écrit le tag du déploiement dans la table
  (dans la même transaction !)

- **Avant chaque upgrade** (before et after), on vérifie le tag en base :
  si il est **≥ au tag actuel**, on sait que **l’upgrade est déjà passé**
  ⇒ on ne fait rien.

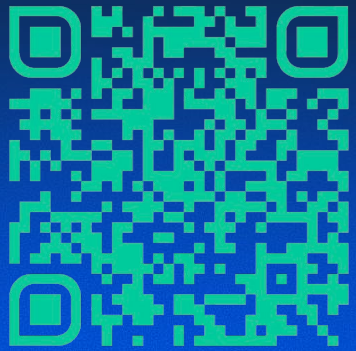
- Éprouvé depuis quelques années ✅

Au programme

- **Migrer le schéma** sans downtime ? ⇒ **Migrations en deux temps**
- **Migrer la donnée** sans pertes ?
- **Assurer l'intégrité** des migrations ?

Retrouvez les diapos et le code :

loic-simon.github.io



Migrer la donnée

```
$ cat app/models/user.py
class User(BaseModel):
    __tablename__ = "user"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str]
    notes: Mapped[str | None]

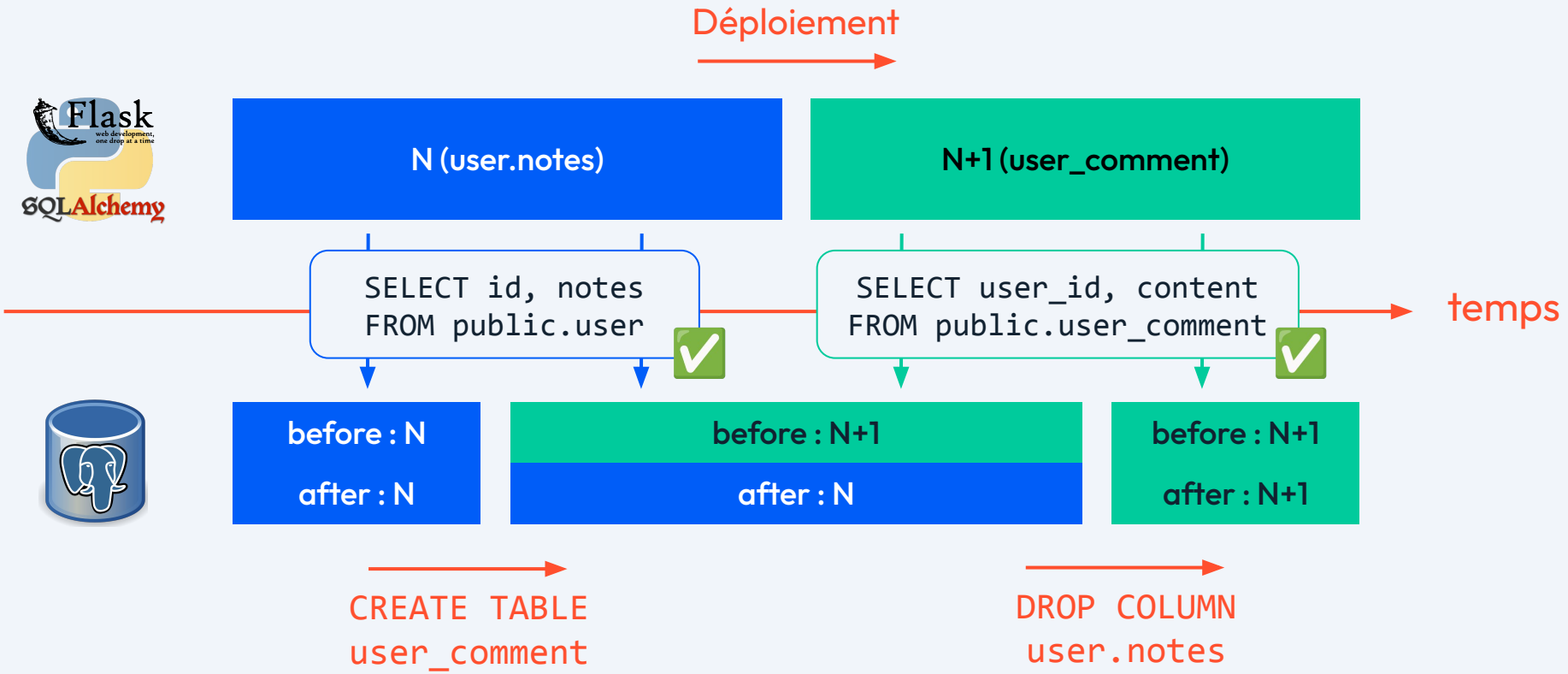
$ cat app/models/user_comment.py
class UserComment(BaseModel):
    __tablename__ = "user_comment"

    id: Mapped[int] = mapped_column(primary_key=True)
    user_id: Mapped[int] = mapped_column(ForeignKey("user.id"))
    content: Mapped[str]
```

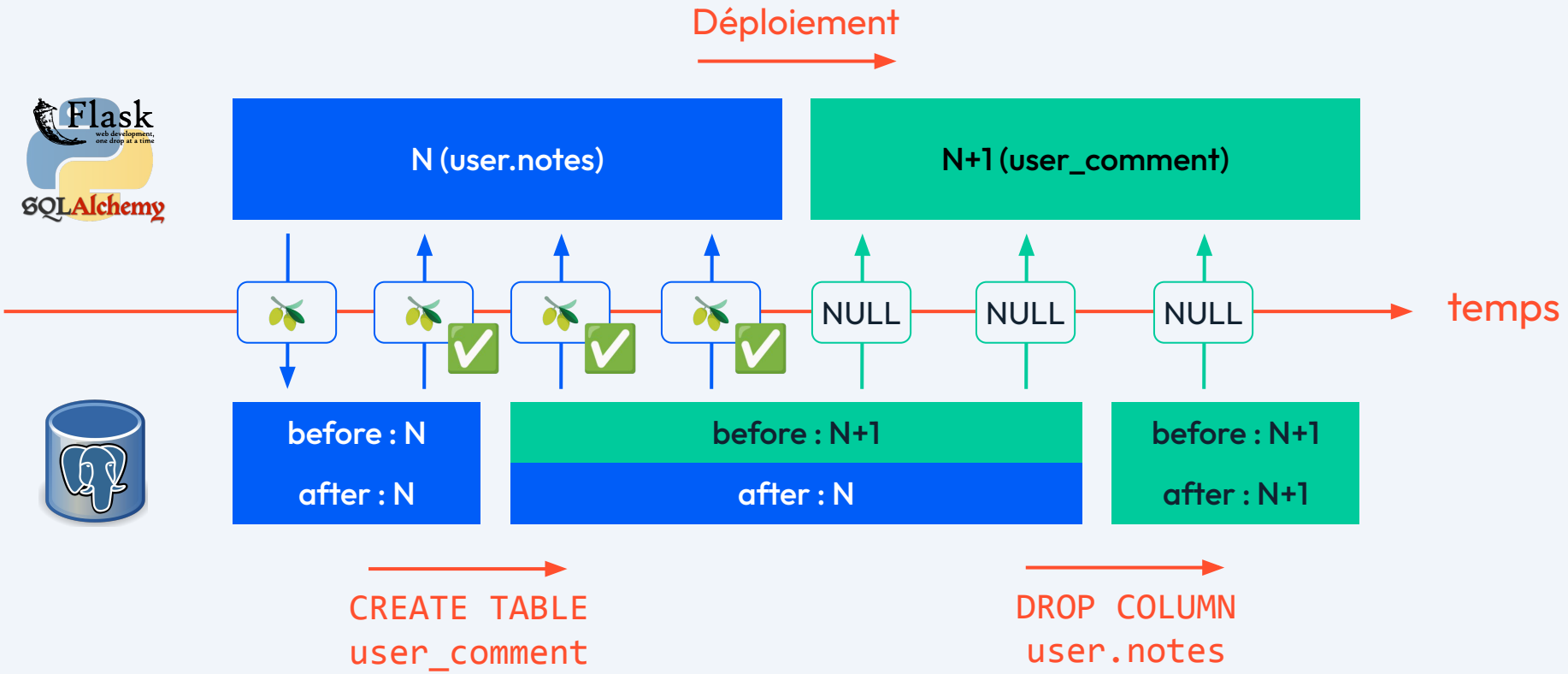
Migrer la donnée

```
$ cat faa622910d09_add_user_comment.py
...
def upgrade(*, step: Literal["before", "after"]):
    if step == "before":
        op.create_table(
            "user_comment",
            sa.Column("id", sa.Integer(), primary_key=True),
            sa.Column("user_id", sa.Integer(), nullable=False),
            sa.Column("content", sa.String(), nullable=False),
            sa.ForeignKeyConstraint("user_id", "user.id"),
        )
    else:
        op.drop_column("user", "notes")
```

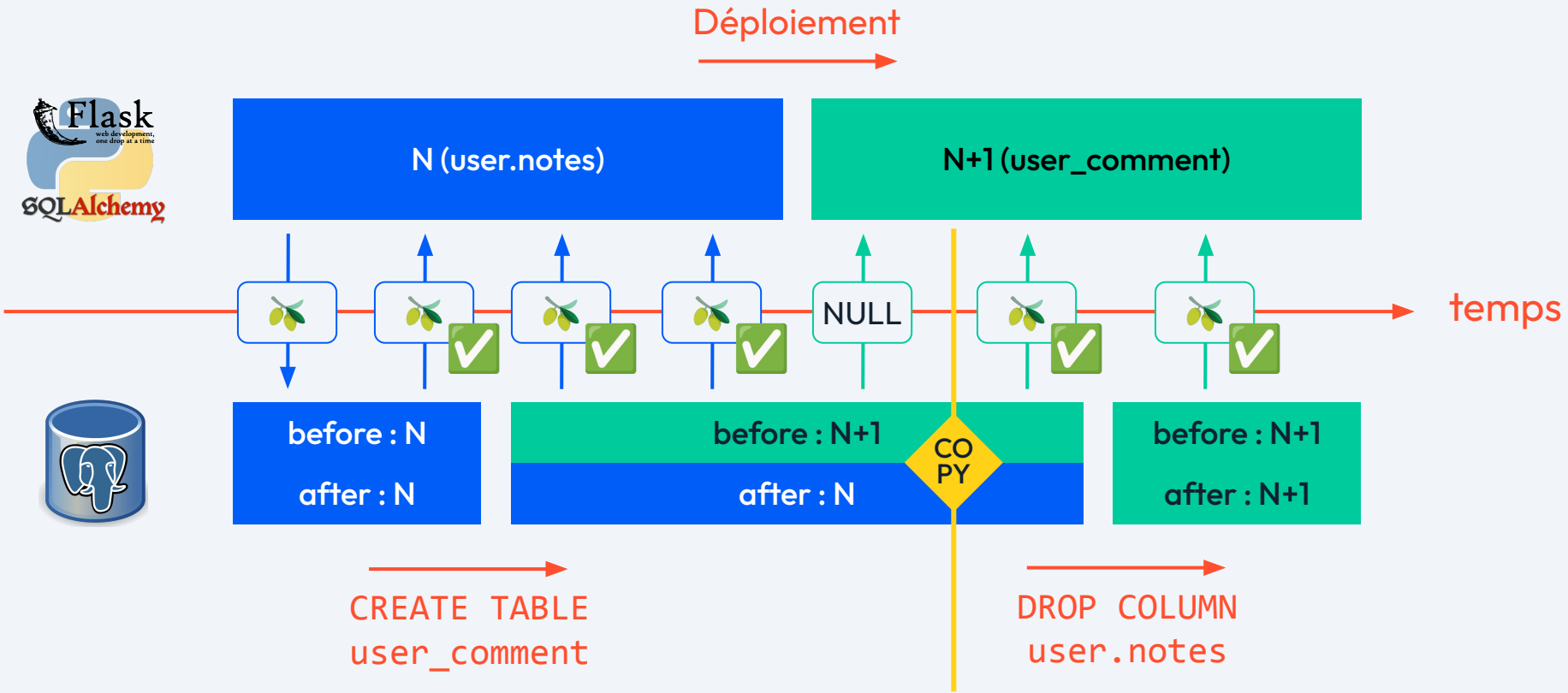
Migrer la donnée



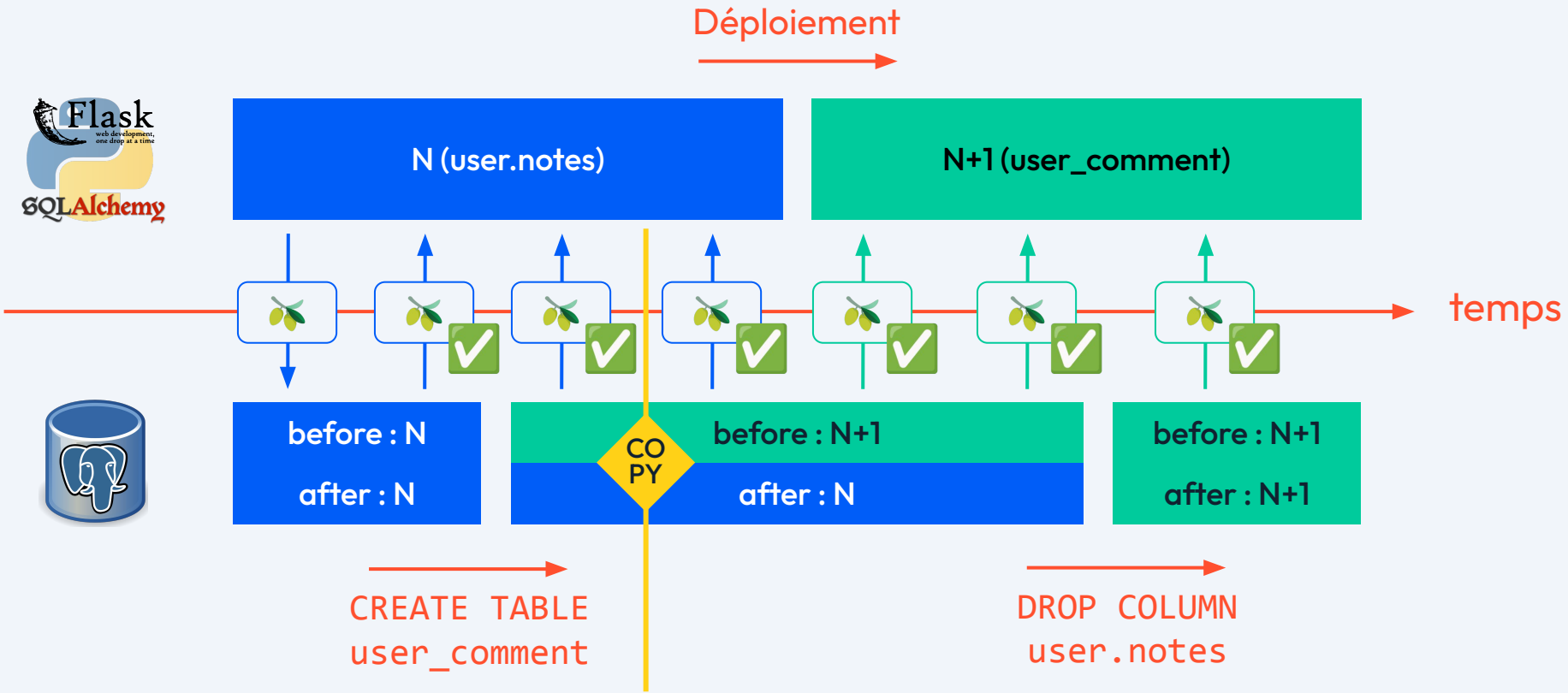
Migrer la donnée



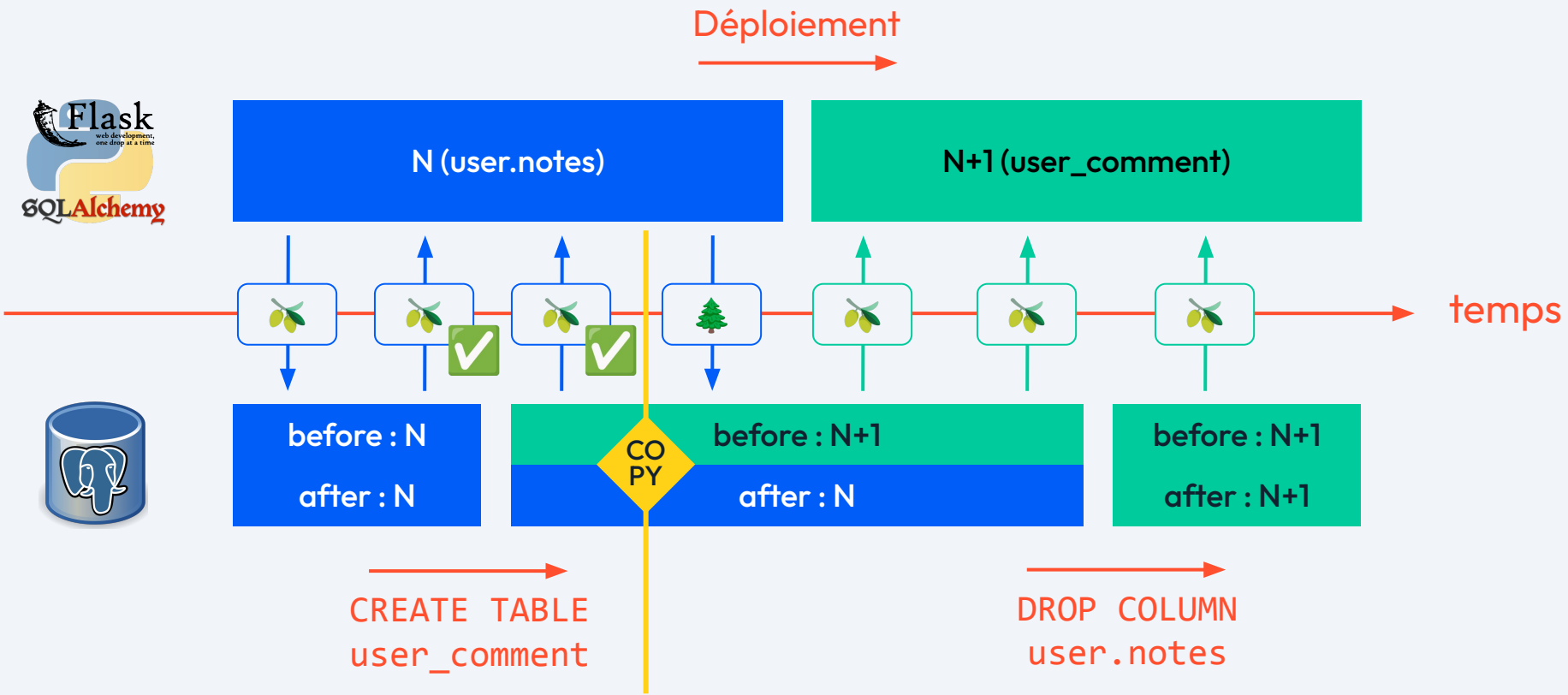
Migrer la donnée



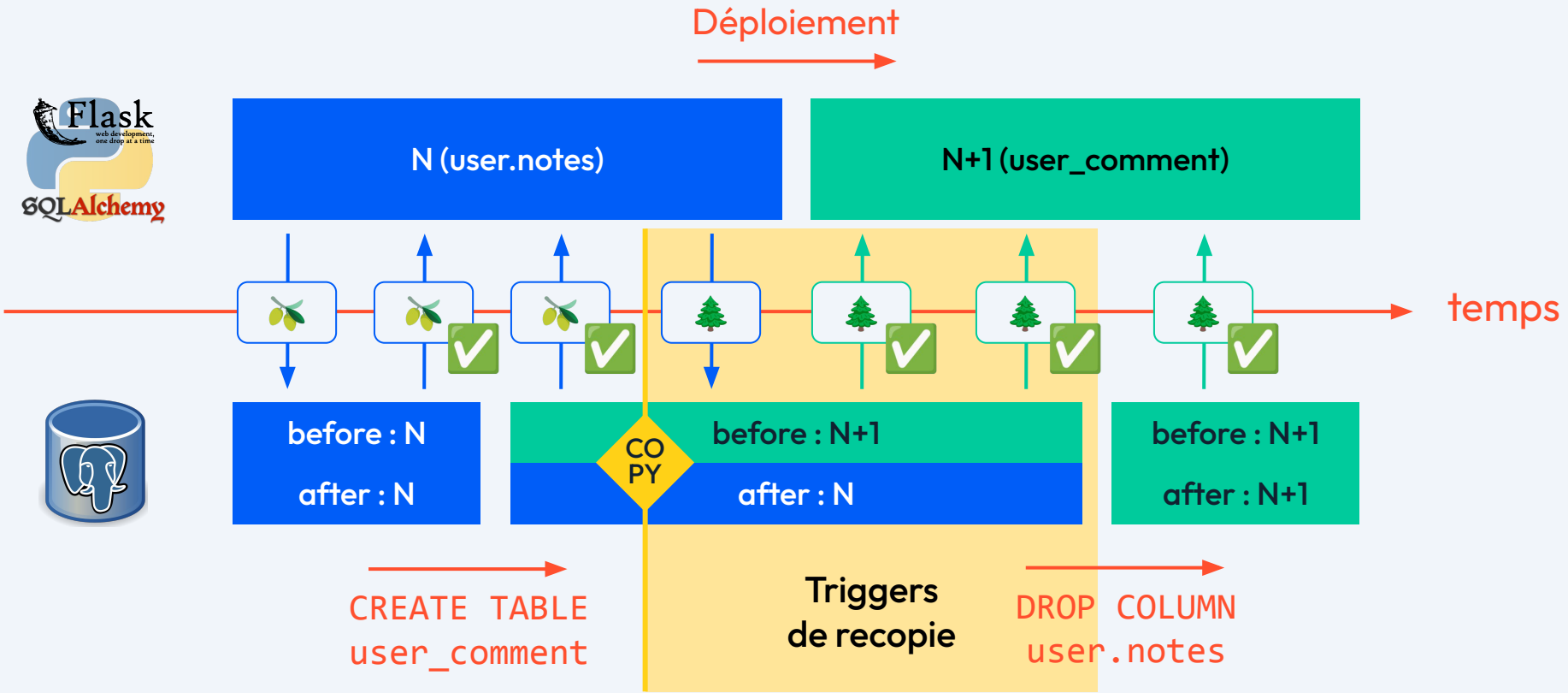
Migrer la donnée



Migrer la donnée



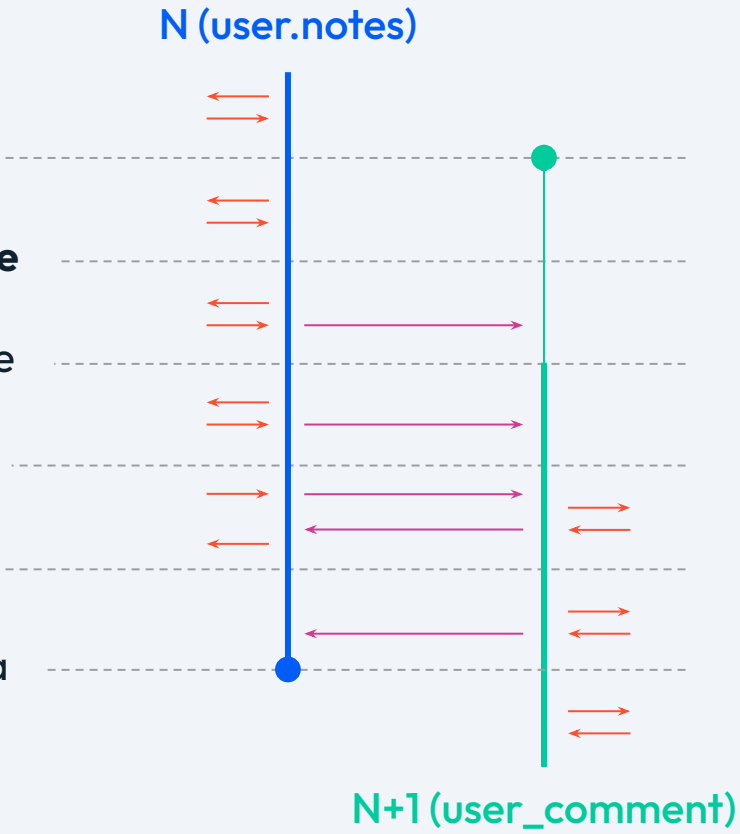
Migrer la donnée



Migrer la donnée : expand/contract

Pattern expand / contract :

1. **Création du nouveau** schéma
2. Création des **triggers de recopie**
3. **Copie de la donnée** préexistante
4. **Déploiement** de la version N+1
5. **Fin du déploiement**
6. **Suppression de l'ancien** schéma & des triggers de recopie



Migrer la donnée : expand/contract

```
CREATE OR REPLACE FUNCTION tmp_copy_notes_to_user_comment_fn() RETURNS TRIGGER AS $fbd$
   BEGIN
       IF OLD.notes IS NOT NULL THEN
           DELETE FROM user_comment WHERE user_id = NEW.id
                                      AND content = OLD.notes;
       END IF;
       IF NEW.notes IS NOT NULL THEN
           INSERT INTO user_comment (user_id, content)
                               VALUES (NEW.id, NEW.notes);
       END IF;
       RETURN NEW;
   END;
   $fbd$ LANGUAGE plpgsql;

CREATE TRIGGER tmp_copy_notes_to_user_comment
AFTER INSERT OR UPDATE OF notes ON public.user
FOR EACH ROW WHEN (pg_trigger_depth() < 1)  -- avoid infinite loops
EXECUTE FUNCTION tmp_copy_notes_to_user_comment_fn();
```

Migrer la donnée : expand/contract

```
CREATE OR REPLACE FUNCTION tmp_copy_user_comment_to_notes_fn() RETURNS TRIGGER AS $fbd$
   BEGIN
       IF TG_OP = 'INSERT' OR TG_OP = 'UPDATE' THEN
           UPDATE public.user SET notes = NEW.content
                              WHERE id = NEW.user_id;
       ELSE -- DELETE
           UPDATE public.user SET notes = NULL
                              WHERE id = NEW.user_id AND notes = OLD.content;
       END IF;
       RETURN NEW;
   END;
   $fbd$ LANGUAGE plpgsql;

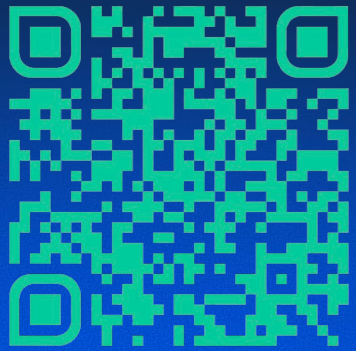

CREATE TRIGGER tmp_copy_user_comment_to_notes
AFTER INSERT OR UPDATE OR DELETE ON public.user_comment
FOR EACH ROW WHEN (pg_trigger_depth() < 1)  -- avoid infinite loops
EXECUTE FUNCTION tmp_copy_user_comment_to_notes_fn();
```

Au programme

- **Migrer le schéma** sans downtime ? ⇒ **Migrations en deux temps**
- **Migrer la donnée** sans pertes ? ⇒ **Pattern expand / contract**
- **Assurer l'intégrité** des migrations ?

Retrouvez les diapos et le code :

loic-simon.github.io



Intégrité des migrations

Avec les migrations en deux temps,

- `alembic upgrade --autogenerate` → un temps ❌
  (et pas infaillible)

Intégrité des migrations

```
$ git diff
--- a/app/models/user.py
+++ b/app/models/user.py
@@ -164,6 +164,7 @@ class User(BaseModel):
     name: Mapped[str]
-    notes: Mapped[str | None]
+    notes: Mapped[str | None] = mapped_column(Computed("'...'"))

$ alembic revision --autogenerate
Generating app/alembic/versions/dd3609e0c585_.py ...  done

$ cat app/alembic/versions/dd3609e0c585_.py
...
def upgrade():
    pass
```

Intégrité des migrations

Avec les migrations en deux temps,

- `alembic upgrade --autogenerate` → un temps ❌
  (et pas infaillible)

- Nécessité de réécriture manuelle → erreurs possibles ⚠️
  (ou framework custom)

Intégrité des migrations

```
$ cat napta_app/migrations/versions/0617559f6f9b_.py
...
def get_high_level_migration() -> HighLevelMigration:
    from napta_app.models import User
    from migrations.high_level import AddColumnMigration

    return AddColumnMigration(User, User.notes)


# -- end of customizable section, do NOT change code below! --


def upgrade(*, step: Literal["before", "after"]) -> None:
    get_high_level_migration()._upgrade(step)
```

Intégrité des migrations

Avec les migrations en deux temps,

- `alembic upgrade --autogenerate` → un temps ❌
  (et pas infaillible)
- Nécessité de réécriture manuelle → erreurs possibles ⚠️
  (ou framework custom)
- Différence schéma théorique / DB → bugs subtils !! 💣
  (non détectables dans les tests)

Intégrité des migrations

```
$ git diff
--- a/app/models/user.py
+++ b/app/models/user.py
@@ -164,6 +164,7 @@ class User(BaseModel):
     name: Mapped[str]
+    notes: Mapped[str | None]

+++ b/app/alembic/versions/1975ea83b712_add_notes.py
@@ -0,0 +1,41 @@
+    ...
+    def upgrade(*, is_after_step: bool):
+        if not is_after_step:
+            op.add_column(
+                "user", sa.Column("notes", nullable=False)
+            )
```

Intégrité des migrations

Avec les migrations en deux temps,

- `alembic upgrade --autogenerate` → un temps ❌
  (et pas infaillible)
- Nécessité de réécriture manuelle → erreurs possibles ⚠️
  (ou framework custom)
- Différence schéma théorique / DB → bugs subtils !! 💣
  (non détectables dans les tests)
- Solution : **test d'intégrité** en CI 😌
  - **Comparaison des schémas** théorique et réel

Intégrité des migrations : comdab

comdab — Compare Database Schemas

python 3.13 | 3.14 | 3.15 pypi v0.4.1 tests passing docs passing types typed

comdab allows you to compare in depth two database schemas to find all differences: missing columns, different nullabilities or defaults, slight changes in function or triggers definitions...

Installation

Use pip to install comdab:

```
pip install comdab
```

Requirements

- Python `>= 3.13`
- sqlalchemy `>= 2.0`

Project links

Data verified by PyPI on Jun 21, 2026 ›

- Homepage
- Issues
- Repository
- Documentation

Key dates

PyPI data ›

Released: Jun 21, 2026

LATEST RELEASE

1 maintainer

PyPI data ›



loic-simon

pypi.org/p/comdab

Intégrité des migrations : comdab

```
$ cat .gitlab-ci.yml
test:backend:migrations-integrity:
  image: [...]/docker-hub/library/docker
  services:
    - name: [...]/docker-hub/library/postgresql:18-3
      variables:
        POSTGRES_DB: naptest
        POSTGRES_PASSWORD: postgres
  script:
    - aws s3 cp s3://napta-test/naptest.tar.gz .
    - tar --extract --file naptest.tar.gz
    - pg_restore --dbname=naptest naptest.dir
    - DB=naptest uv run alembic upgrade
    - DB=naptest MIGRATION_STEP=after uv run alembic upgrade
    - uv run check_migrations.py
```

Intégrité des migrations : comdab

```
$ cat check_migrations.py
with (
    create_engine(get_db_url("naptest")).connect() as naptest_conn,
    connect_to_tmp_database() as schema_conn
):
    BaseModel.metadata.create_all(schema_conn)

    reports = comdab.compare_databases(
        left_connection=schema_conn,
        right_connection=naptest_conn,
        rules={comdab.ROOT.tables["alembic_.+"]: "ignore"},
    )
    if reports:
        print(reports)
        sys.exit(1)
```

Intégrité des migrations : comdab

```
$ python check_migrations.py
[
    ComdabReport(
        level='error',
        path=ROOT.tables['user'].columns['notes'].nullable,
        left=True,
        right=False,
    )
]
```

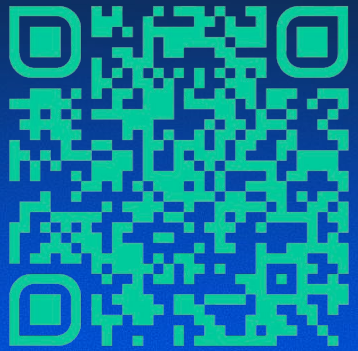
- Détecte les différences dans les tables, colonnes, contraintes, index, vues, séquences, types customs, fonctions, triggers…
- Système de rules pour ignorer certaines différences

Conclusion

- **Migrer le schéma** sans downtime ? ⇒ **Migrations en deux temps**
- **Migrer la donnée** sans pertes ? ⇒ **Pattern expand / contract**
- **Assurer l'intégrité** des migrations ? ⇒ **comdab**

Retrouvez les diapos et le code :

loic-simon.github.io



Merci !

Des questions ?

NAPTA On recrute !

- Stage Développeur·euse Python / Data Engineering - 6 mois
- Senior Fullstack Developer | Next.js / Python
- Lead Front-End (Fullstack) | Next.js / Python

Welcome to the Jungle

wttj.com/companies/napta